TD 01: Grammaires (informelles)

On considère la grammaire suivante.

Les littéraux.

VARIABLES	$x = [a-z]^+$
NOMBRES	$n = [0-9]^+$
CHAÎNES	$s =$ "$[\sim$ "$]^*$"

Les non-terminaux.

TYPE	$\tau ::=$	`Num`	(nombres)
		`Str`	(chaînes)
		$\tau_1 \to \tau_2$	(fonctions)
		$(\tau_1, \ldots, \tau_n)$	(produit)
		`Unit`	(produit vide)

EXPRESSION	$e ::=$	x	(variable)
		n	(nombre littéral)
		s	(chaîne littérale)
		$e_1 + e_2$	(ajouter)
		$e_1 - e_2$	(ajouter)
		$e_1 * e_2$	(multiplier)
		e_1 / e_2	(diviser)
		e_1 ^ e_2	(concaténer)
		$(e_1, \ldots, e_k)$	(tuple)
		`unit`	(tuple vide)
		`let` x=e_1 `in` e_2	(définition)
		e_1 e_2	(application)
		`fun` $(x{:}\tau)$ $\Rightarrow$ e	(abstraction)
		`if` $e_1 \geq 0$ `then` e_2 `else` e_3	(test)

Exercice 1 (Arbres et priorités) :

Question 1 – Écrire l'arbre syntaxique des expressions suivantes. Expliciter les priorités dont vous avez besoin.
- `let x = "hello" in x^x`
- `fun (f : Str → Num) ⇒ f "test"`

Question 2 – Écrire l'AST des expressions suivantes. Expliciter les associativités et priorités dont vous avez besoin.
- `let toto = "hello" in x*x-32*x-79`
- `fun (f : Str → Num → Num) ⇒ f "test" 3`

Exercice 2 (Étendre la grammaire) :

Question 1 – Les ... ne seraient pas acceptés dans une grammaire formelle. Trouver une manière d'exprimer le produit formellement.

Question 2 – Le test est limité à la comparaison avec 0. Généraliser le test en ajoutant les booléens avec (certains de) leurs opérateurs.

Question 3 – En OCaml, les définitions `let...in` sont plus générales : on peut remplacer la variable x par un motif (*pattern* en anglais), qui est soit une variable, soit un tuple de motifs (qui peut être vide), soit un joker `_`. Généraliser le `let...in`.